Lab 02 – Inject and Detect a Stuxnet-style Logic Bomb

Industrial Security (Master) • Maps to Section 02 • 2–3 hours

LAB 02

Goal

Plant a small "stealth" modification on a soft-PLC you own. Detect it from an external monitoring point. End with a written disclosure-ready report.

Step 1 – Baseline

```
docker compose up -d            # bachelor/labs/_stack
# Upload conveyor.st via OpenPLC web UI (Lab 02 of bachelor course).
# Capture baseline blocks:
docker compose exec student python3 - <<'PY'
import hashlib, snap7
c = snap7.client.Client(); c.connect("172.28.0.10", 0, 1)
for t, n in [(snap7.types.Block.OB, 1)]:
    b = c.upload(t, n); print(f"OB{n} sha256: {hashlib.sha256(b).hexdigest()}")
c.disconnect()
PY
```

Step 2 – Plant the modification

Edit the ladder so an unused input (`IX0.4`) gates a counter that, after N=50 cycles, drives `Motor` high regardless of the start/stop logic. Download *only* the modified runtime, do not save the project. Re-take SHA-256.

Step 3 – Watch from the outside

```
docker compose exec student python3 /labs/scripts/modbus_poll_loop.py
```

Observe `Motor` firing without operator action. Capture in Wireshark; identify the function code(s).

Step 4 – Detection

Re-compute the SHA-256 from the PLC after the modification. Compare to baseline. Document how a continuous-monitoring product (Claroty, Dragos, Nozomi) would surface this within seconds.

Deliverable

A 1-page report containing: baseline hash, post-injection hash, PCAP of the unauthorised `Motor` firing, a sentence on the mitigation that would have prevented the download (e.g. key-switch in RUN), and the disclosure email you would send a vendor.

Caution

Do this only on your lab PLC. Modifying production ladder logic without written authorisation is criminal in essentially every jurisdiction.